MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Phase 4 — Capstone

Capstone Project Brief

Trainee Handbook — everything you need to know before you start building

---

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Phase 4 — Capstone

Capstone Project Brief

Trainee Handbook — everything you need to know before you start building

Sessions	S25 (Kickoff, offline) · S26 · S27 · S28 (online gates) · S29 (Final presentation, offline)
Build window	≈ 2.5 weeks — most of the work happens at home
Teams	3 teams × 3 trainees, one independent Git repository per team
Weight	35% of the final course grade (Team 70% + Individual 30%)
Trainer	Dr. ElSayed Mohamed Elsayed Baladoh

مرحباً بكم في مرحلة الCapstone

في هذه المرحلة لا نتعلّم أنماطاً جديدة؛ نبني المنتج كاملاً من البداية بالشكل النهائي، بفريقنا، ونُدافع عن كل قرار اتخذناه .هذا الملف هو مرجعكم الكامل : ماذا نبني، بأي تقنيات، بأي ترتيب، وكيف سيتم التقييم .اقرأه كاملاً قبل جلسة الكيك-أوف، وارجع إليه طوال فترة البناء.

How to use this document

- Sections 1–3: what we are building and why, and the calendar.
- Sections 4–6: the requirements, the architecture and the technology choices (what is fixed and what you decide).
- Sections 7–9: the build plan (layers and gates), the Starter Repo, and the Bonus Features.
- Sections 10–12: how teams work, what you must deliver, and how you are graded.
- Sections 13–14: the final presentation, and your checklist for Session 25.
- Keep it open while writing your ADD (Architecture Decision Document) — it tells you what a good one looks like.

1. Vision & Product

Vision

"Rebuild the platform we learned — once, in its final form — into a product a stranger can run, deploy, load-test and question."

Product: Enterprise E-Commerce Platform. Customers browse a catalogue, place orders, and are notified. Admins manage the catalogue and stock.

What your team delivers: one independent Git repository containing 8 platform services, infrastructure, Kubernetes / Helm / GitOps assets, observability, load tests, documentation — plus one Bonus Feature.

Why a clean rebuild instead of continuing the labs?

During the course, the goal was to practise each topic. Some components were built one way and rebuilt another way a session later, and labs were sometimes finished late. That was right for learning and wrong for building a product. In the Capstone every team starts from the same Starter Repo and builds each component once, in its final shape — so nobody is penalised for late labs, and everybody ends with a complete, working product.

Aspect	Lab Era (Sessions 1–24)	Capstone (Sessions 25–29)
Repository	One personal repo, evolved lab by lab	One repository per team, from the Starter Repo
Security	JwtAuthFilter (S3), later Resource Server (S20)	Keycloak Resource Server from day one — build once
Stock data	In-memory map (S6), later a DB	PostgreSQL + Flyway from day one
Saga	Choreography (S7), later Orchestration (S12)	ONE style, chosen by your team, justified in the ADD
Debt fixes	Outbox / Idempotency / Versioning added in S22	Built-in from the first commit
Guidance	One lab at a time	Full picture first: requirements, architecture, technologies
Work mode	Individual labs	Team of 3, rotating roles, PR review, git log evidence

2. Calendar & What Happens in Each Session

The five sessions are checkpoints (“gates”). The building itself happens in your own time between them. Plan roughly 2 hours per day per person.

Session	Format	What happens	Your deliverable
S25 — Kickoff	Offline, 5h	Project Charter, team formation, Capstone Architecture Review, Sprint 0	Repo + L0 running, ADD (peer-reviewed), backlog, Team Charter
S26 — Gate G1	Online, 2.5h	Live review of your repos; PR reviews; unblock issues	Layers L1 + L2 working
S27 — Gate G2	Online, 2.5h	Review of Saga + deployment; security hardening feedback	Layers L3 + L4: running in a Kubernetes cluster
S28 — Gate G3	Online, 2.5h	Review of load tests and the bottleneck you found and fixed	Layer L5: k6 results + Performance Report
S29 — Final	Offline, 5h	Team presentations, live demo, individual oral Q&A, graduation	Layer L6: Bonus, docs, demo; final repo + report + slides

Note on the windows: the gaps between sessions are uneven (about 5, 2, 5 and 2 days). Use the long windows for the heavy layers (L3 Saga + Outbox, L4 Kubernetes + GitOps) and the short windows for tests, fixes and documentation. Start L3 and L4 early.

3. Functional Requirements

MoSCoW priority: **M** = Must (Core), **S** = Should, **C** = Could. Completing all Must and Should requirements gives you a complete Core product.

ID	Requirement	Pri	Owner service(s)
FR-01	Anyone can browse products (paginated, public, no token)	M	Product, Gateway
FR-02	Admin can create, update, delete products (ADMIN role only)	M	Product
FR-03	Product detail includes the category name (read model, not a raw entity)	S	Product
FR-04	Customers sign in through Keycloak; protected routes reject missing/invalid tokens	M	Gateway, Keycloak
FR-05	Authenticated customer places an order; the API answers immediately with orderId and status PENDING	M	Order
FR-06	Order checks stock synchronously before accepting; unavailable stock is rejected without touching payment	M	Order, Inventory
FR-07	Stock is reserved for the order and released if the order fails	M	Inventory
FR-08	Payment is processed exactly once per order — a retried request never charges twice (idempotency)	M	Payment
FR-09	Order reaches CONFIRMED on payment success; on payment failure it reaches CANCELLED and stock is released (compensation)	M	Order, Inventory, Payment
FR-10	Customer can read order status by id; and list own orders only	M / S	Order
FR-11	Customer receives a confirmation on CONFIRMED and a notice on CANCELLED; failed sends are retried, then parked in a Dead Letter Topic	M	Notification
FR-12	Admin can view and adjust stock levels	S	Inventory
FR-13	Public endpoints are rate limited per client	S	Gateway (+ Redis)
FR-14	Service-to-service calls outside a user request authenticate with Client Credentials	C	Order, Inventory, Keycloak
FR-15	Product reads are cached and evicted on every write (list + item)	S	Product, Redis
FR-16	Your team delivers one Bonus Feature (Section 9)	M	Team choice

Non-Functional Requirements

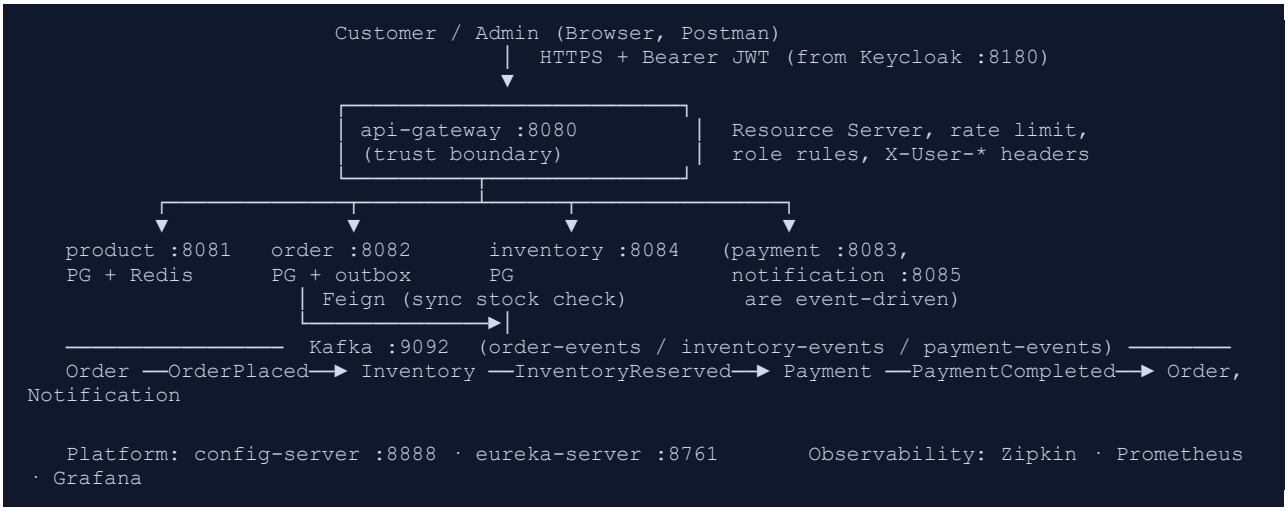
ID	Requirement	How it is verified
NFR-01	Availability: if Payment is down, orders are not lost — they stay PENDING or are cancelled with compensation	Chaos test: stop payment-service, place an order
NFR-02	Latency: GET /api/v1/products P95 < 200 ms (cached); POST /api/v1/orders P95 < 800 ms at 20 VUs	k6 report (Gate G3)
NFR-03	Throughput: Gateway sustains ≥ 50 req/s on reads. You may propose a different target in your ADD	k6 report
NFR-04	Security: no secrets in Git, JWT validated at the Gateway, non-root containers, least-privilege ServiceAccount per service	CI guards + review
NFR-05	Consistency: no orphaned stock reservation older than 30 seconds after an order is CANCELLED	Test + query

ID	Requirement	How it is verified
NFR-06	Observability: one traceId across HTTP and Kafka hops; JSON logs carry traceId	Zipkin trace screenshot
NFR-07	Testing: ≥ 60% line coverage on service layers; ≥ 1 TestContainers test per database-owning service	JaCoCo in CI
NFR-08	Deployability: docker compose up runs the platform locally; helm install runs it on Kubernetes	Live demo
NFR-09	Maintainability: Flyway migrations; every public API under /api/v1	Code review
NFR-10	Messaging: at-least-once delivery + idempotent consumers + DLT for exhausted retries	Test: redeliver an event

4. Target Architecture

Service	Port	Database / Store	Key responsibilities	Layer
config-server	8888	Git config repo	Central configuration, environment profiles	L0
eureka-server	8761	—	Service discovery	L0
api-gateway	8080	Redis (rate limit)	Single entry, OAuth2 Resource Server (Keycloak), routing, rate limiting, role rules	L1
product-service	8081	PostgreSQL + Redis cache	Catalogue CRUD, read model (projection), cache-aside, /api/v1	L1
order-service	8082	PostgreSQL (+ outbox)	Place order, stock pre-check (Feign + Resilience4j), Saga initiator, order status	L2/L3
payment-service	8083	PostgreSQL (payments + idempotency)	Process payment once, refund, failure-rate switch for tests	L2/L3
inventory-service	8084	PostgreSQL	Stock, reserve, release, low-stock query	L2/L3
notification-service	8085	—	Kafka consumer, @RetryableTopic, DLT, health	L3

Infrastructure (comes with the Starter Repo): PostgreSQL 5432 · Kafka 9092 (+ Zookeeper 2181) · Redis 6379 · Keycloak 8180 · Zipkin 9411 · Prometheus 9090 · Grafana 3000. You may not add infrastructure beyond this list unless your ADD names a real problem that justifies it.



Two paths, two purposes: the SYNC path (Order → Inventory over Feign) answers "can I accept this order right now?" — the caller needs the answer. The ASYNC path (Saga over Kafka) carries the business transaction — the caller does not wait.

The Place-Order flow — happy path and failure paths

Happy Path

1. POST /api/v1/orders + JWT → Gateway validates
2. Order: Feign stock check (CB/Retry/Bulkhead/TimeLimiter)
3. Order saves PENDING + outbox row (ONE transaction)
4. Publisher sends OrderPlaced → Kafka
5. Inventory reserves stock → InventoryReserved

Failure Paths

No stock (step 2): REJECTED, payment never touched

Payment fails (step 6): PaymentFailed → Inventory releases stock → InventoryReleased → Order CANCELLED → Notification sends cancel notice

Payment service down: Circuit Breaker / retries per your design; the order is never lost

Duplicate event delivered: consumers check state first;

6. Payment charges once → PaymentCompleted 7. Order → CONFIRMED · Notification sends confirmation The client got orderId + PENDING at step 3 and polls GET /orders/{id}	Payment uses orderId as idempotency key **Consumer keeps failing:** @RetryableTopic → .DLT topic → alert log

API surface (all public APIs under /api/v1)

Service	Endpoint	Access
product	GET /api/v1/products · GET /api/v1/products/{id}	Public
product	POST / PUT / DELETE /api/v1/products[/{id}]	ADMIN
order	POST /api/v1/orders · GET /api/v1/orders/{id} · GET /api/v1/orders (own)	CUSTOMER
inventory	GET /api/v1/inventory/check?productId&quantity (internal, via Feign)	Authenticated service
inventory	GET / PUT /api/v1/inventory/{productId}	ADMIN
payment	POST /api/v1/payments (Idempotency-Key header) · POST /api/v1/payments/{id}/refund	Internal / ADMIN
all	GET /actuator/health · /actuator/prometheus	Internal

Event catalogue (Choreography default): OrderPlaced · InventoryReserved · InventoryReservationFailed · PaymentCompleted · PaymentFailed · InventoryReleased · OrderConfirmed · OrderCancelled. Topics: order-events, inventory-events, payment-events. If your team chooses Orchestration, replace these with commands + results topics and record the change in the ADD.

5. Technology Decision Matrix — what is fixed, what you decide

Level	Meaning	Rule
? Fixed	Not negotiable — the platform depends on it	No ADD needed
? Default	Recommended choice; you may replace it	Replacement requires a written justification in ADD §5–§7
? Open	Your team decides	Decision and reason recorded in the ADD

Layer	Level	Default technology	Allowed alternatives (justify in ADD)
Language / framework	?	Java 21, Spring Boot 3.x, Spring Cloud, Maven multi-module	—
Service discovery	?	Eureka (separate service)	Kubernetes DNS only, if you also drop Eureka in Compose and explain client-side load balancing
Configuration	?	Config Server (separate service)	ConfigMap/Secret only in Kubernetes + env vars in Compose
Identity & security	?	Keycloak (realm ecommerce-platform), Gateway as OAuth2 Resource Server	—
Data storage	?	PostgreSQL, database-per-service, Flyway migrations	—
Cache	?	Redis, cache-aside, evict on every write	Caffeine (single-instance only — explain multi-replica impact)
Messaging	?	Kafka	—
Saga style	?	Choreography (default for 3 participants)	Orchestration — ADD must state where Saga state is stored and why Choreography was not enough
Sync communication	?	OpenFeign	WebClient (only if the calling service is reactive)
Resilience	?	Resilience4j on Order → Inventory (CB, Retry, Bulkhead, TimeLimiter) + Payment path	Istio outlier detection in addition (never instead of business fallbacks)
Event publishing	?	Transactional Outbox with polling publisher	Direct publish with an idempotent consumer — ADD must state the failure window you accept
Read model	?	Spring Data projection + service split	Full separate read store (needs a justified problem)
Testing	?	JUnit 5, Mockito, @WebMvcTest, TestContainers, JaCoCo	Pact contract test (recommended for order↔inventory), WireMock
Observability	?	Micrometer Tracing + Zipkin, Prometheus, Grafana, JSON logs	—
Containers	?	Docker multi-stage, non-root user, healthcheck	—
CI	?	GitHub Actions: test gate → build → push to GHCR (sha tag)	—
Deployment	?	Kubernetes + Helm chart per service	Raw manifests for some services (Helm required for at least two)

Layer	Level	Default technology	Allowed alternatives (justify in ADD)
GitOps	⍰	ArgoCD Application (automated, selfHeal, prune)	Helm-only deploy — explain how drift is detected
Service mesh	⍰	Not required	Istio mTLS (Advanced Scope) — only with a named problem
Load testing	⍰	k6 (smoke → load → stress)	—
Not allowed	—	Elasticsearch, extra databases, extra message brokers, Axon, Event Sourcing	Out of scope for this Capstone

Decision guide 1 — ⍰ Saga style: Choreography vs Orchestration

The decision is yours. The reasoning must be written in your ADD, together with what would make you switch.

⍰ DESIGN CHOICE: Saga style for the Order → Inventory → Payment flow

3 participants, linear flow, no conditional branching? → Choreography (simpler, no coordinator to keep alive)

You need to see "where is order X right now?" from one place? → Orchestration (state in one table) OR Choreography + order status + tracing

Business rules will branch (fraud check, loyalty, partial refunds)? → Orchestration (rules live in one class)

Team new to event-driven design? → Choreography first, add tracing early

Orchestrator state must survive a restart? → Persist SagaState in PostgreSQL (in-memory state is known technical debt)

Different teams deploy services on different schedules? → Choreography (loose coupling)

Cannot decide? → Choreography + Outbox + idempotent consumers, then write down what would make you switch

Choosing Orchestration "because it is more advanced" is not a reason and will not pass the ADD review.

Decision guide 2 — ⍰ Repository & branching inside a team

⍰ ENGINEERING DECISION: one monorepo vs one repo per service

Team of 3, 8 services, shared parent POM, one CI, one demo? → ONE monorepo per team (the Starter Repo layout)

Branching: short-lived branches (max 1–2 days) merged by Pull Request into main; never commit directly to main after L0

Every PR needs one review from a teammate (the rotating Tech Lead decides on disagreement)

Commit convention: capstone-Lx: short-description (example: capstone-L2: add-payment-idempotency)

Pair work: add Co-authored-by: Name <email> so the git log stays honest for the Individual assessment

6. Build Plan — Layers, Gates and Time

Each layer ends with something that RUNS. A late layer never blocks an earlier one.

Layer	Content	Definition of Done (must be demonstrable)	Gate
L0	Repo from Starter, infra up, Config + Eureka running	docker compose up: all infra healthy; config-server and eureka-server registered; verify-l0.sh passes	S25
L1	Product + Gateway + Keycloak security	Public GET works without token; POST without token = 401; ADMIN-only rules; Flyway; Redis cache with eviction; /api/v1	G1 — S26
L2	Inventory + Payment + Order (sync part)	Order pre-checks stock via Feign with Resilience4j; Payment idempotent per key; each service has tests	G1 — S26
L3	Saga + Outbox + Notification	Happy path CONFIRMED; payment failure → CANCELLED + stock released; DLT works; consumers idempotent	G2 — S27
L4	Docker, CI, Kubernetes, Helm, GitOps + hardening	All images built by CI; platform runs in a cluster; probes, Secrets, ServiceAccount; ArgoCD Synced/Healthy	G2 — S27
L5	Observability + k6 + optimization	One trace across HTTP + Kafka; Grafana panel; k6 smoke/load/stress results; one bottleneck found and fixed; Performance Report	G3 — S28
L6	Bonus Feature + docs + demo	Bonus Core delivered; README, final ADD, architecture diagrams; 15-minute demo rehearsed	G4 — S29

Time budget	Estimate
Total team effort L1–L6	≈ 100–120 hours (≈ 35–40 hours per trainee over ≈ 2.5 weeks, about 2 hours per day)
Heaviest layers	L3 (Saga + Outbox) and L4 (Kubernetes + GitOps) — start early
What the Starter Repo saves you	≈ 20 hours of boilerplate: parent POM, module skeletons, Dockerfile template, Compose, CI template, Keycloak realm, Prometheus config

Gate status rule — Green / Amber / Red

Green: the layer's Definition of Done is demonstrated live.

Amber: partly done, with a written recovery plan that fits before the next gate. Allowed once per team.

Red: no demonstrable result. The trainer schedules a 30-minute intervention; you apply the scope-cutting order below.

If you are behind, cut in this order: Stretch goals → Bonus Could stories → FR-14 → FR-13 → FR-12 → Bonus Core scope (reduce, never drop).

Never cut: tests, security rules, the compensation path, deployment.

Tell the trainer early. An Amber with a plan is fine; a silent Red is not.

7. The Starter Repo

You receive a GitHub Template Repository. Click “Use this template” to create your team repository (capstone-team-a / -b / -c), add your teammates and the trainer, and protect main.

```
capstone-platform/                          (one repository per team)
├── README.md                               how to run, layer checklist, team info
├── pom.xml                                 parent POM: Java 21, Spring Boot + Cloud BOMs,
JaCoCo, plugins
├── platform/
│   ├── config-server/                      READY  (working, reads /config-repo)
│   ├── eureka-server/                      READY  (working)
│   └── api-gateway/                        SKELETON (pom, main class, actuator, Dockerfile) —
you build
├── services/
│   ├── product-service/                    SKELETON
│   ├── inventory-service/                  SKELETON
│   ├── order-service/                      SKELETON
│   ├── payment-service/                    SKELETON
│   └── notification-service/               SKELETON
├── config-repo/                            application.yml + per-service yml placeholders
├── deployment/
│   ├── docker/docker-compose.yml           READY: postgres(+init dbs), kafka, zookeeper, redis,
keycloak,
│   │                                       zipkin, prometheus, grafana, all with healthchecks
│   ├── docker/keycloak/realm-export.json   READY: realm, gateway client, roles ADMIN/CUSTOMER,
2 test users
│   ├── docker/prometheus/prometheus.yml    READY: scrape targets prepared
│   ├── kubernetes/                         EMPTY + README (you build at L4)
│   ├── helm/                               EMPTY + README
│   └── argocd/                             EMPTY + README
├── k6/                                     smoke-test.js template only
├── docs/
│   ├── CHARTER.md                          the project charter
│   ├── adr/ADD-TEMPLATE.md                 8-section template
│   ├── BACKLOG-TEMPLATE.md                 story format + estimate scale
│   └── architecture/                       put your diagrams here
├── scripts/verify-l0.sh                    checks infra health + config + eureka
└── .github/
    ├── workflows/ci-template.yml           test → build → push (you complete)
    ├── PULL REQUEST TEMPLATE.md
    └── CODEOWNERS
```

8. Bonus Features — each team owns ONE primary Bonus

Bonus Features are part of the product, not a separate track. Each team drafts a different primary Bonus in S25 (ties: the team with the lower Skills Matrix average gets priority, otherwise a coin flip). Extra Bonus features are optional and earn credit only after your Core and primary Bonus are complete. Elasticsearch and any other new infrastructure are out of scope.

Bonus B1 — Product Reviews & Ratings (Tier 1 · new service review-service :8086)

Business problem: shoppers hesitate without social proof; the catalogue shows no ratings.

Core (mandatory): authenticated customer submits one review per product (rating 1–5 + text); GET reviews paginated; product detail shows average rating and count, updated through an event (ReviewSubmitted → product-service read model). Own PostgreSQL + Flyway; own Helm chart and CI.

Stretch: verified-purchase badge (Feign to Order, with Circuit Breaker); admin moderation; review edit/delete with correct average recalculation.

Patterns reused: S6 Feign, S7 events, S18 read model, S22 idempotent consumer.

Main risks: average rating drifting under duplicate events; new service missing from Gateway routes and Helm.

Bonus B2 — Order Analytics Dashboard (Tier 1 · CQRS + Grafana · no new infrastructure)

Business problem: operations cannot see order volume, revenue or Saga failures without querying databases.

Core: a read model built from order events (orders per status per hour, revenue per hour, cancelled ratio); GET /api/v1/analytics/summary (ADMIN); Grafana dashboard with at least 3 panels (orders per minute by status, revenue, Saga failure rate).

Stretch: top-N products; alert rule on cancelled-ratio; export CSV.

Patterns reused: S17 metrics, S18 CQRS, S22 idempotency. Placement (inside order-service read side or a small analytics module) is □ Open — decide in the ADD.

Main risks: double counting on redelivered events — the projection must be idempotent.

Bonus B3 — Multi-Tenant Gateway (Tier 2 · hardest conceptually)

Business problem: two shops want to run on one platform with separate catalogues and separate limits.

Core: two tenants; tenant resolved at the Gateway (JWT claim tenant_id — decision recorded in ADD); tenant context forwarded as a header; Product data isolated with a tenant_id column enforced in the service; tenant A cannot read or change tenant B's products (proved by tests); per-tenant rate limit.

Stretch: per-tenant configuration; tenant-aware cache keys; tenant in log lines and traces.

Patterns reused: S2–3 Gateway, S19–20 Keycloak claims, S8 cache keys.

Main risks: data leakage through cache keys or missing filters — your ADD must list every place tenant id must appear.

Bonus B4 — Real-Time Inventory Alerts (Tier 2 · Kafka + SSE/WebSocket)

Business problem: admins learn about low stock too late.

Core: inventory-service publishes LowStock when available quantity falls below a configurable threshold; an admin client connected through the Gateway receives the alert within 2 seconds (Server-Sent Events or WebSocket — □ Open, justify in ADD); alerts deduplicated per product.

Stretch: alert acknowledgement; alert history endpoint; behaviour with two replicas of the pushing service.

Patterns reused: S7 Kafka, S13 retry/DLT, S20 security on the streaming connection.

Main risks: authentication of a long-lived connection through the Gateway; fan-out with more than one instance.

9. Teams, Roles and Ways of Working

- **3 teams of 3.** In S25 everyone fills a Skills Matrix (1–5 on Spring Boot, Kafka/Saga, Docker/K8s/CI, Testing, Security/Keycloak); the trainer balances the teams so that strong members are spread.
- **Trainees who are behind on labs** are paired with a strong partner and use the same Starter Repo — nobody starts from an old lab repository.
- **Roles rotate at every gate** (G1→G2→G3→G4): **Tech Lead** (architecture, PR decisions), **DevOps/Security** (CI, Docker, Kubernetes, Keycloak), **QA/Docs** (tests, load test, README, ADD upkeep). Everyone holds every role at least once.
- **Team Charter (one page, signed):** working hours, communication channel, review SLA (PR reviewed within 24 hours), how disagreements are settled, what to do when a member is blocked.
- **Git rules:** main is protected; short-lived branches; every change through a Pull Request with one teammate review; commit messages follow capstone-Lx: short-description; pull main daily; never reformat shared files (parent POM, docker-compose).
- **Individual evidence:** each member must have commits in at least 3 different services. Pair work is fine — use Co-authored-by trailers.
- **No secrets in Git**, ever. Use environment variables, Kubernetes Secrets and CI secrets.

10. What You Must Deliver

10.1 Architecture Decision Document (ADD) — 8 mandatory sections

```
# docs/adr/ADD-<team>.md

1. PROBLEM STATEMENT        who is the user, what is the pain, how will we measure success
2. BOUNDED CONTEXT          what our Bonus owns / does not own; data ownership; new or existing
service
3. API CONTRACT + EVENTS    endpoints (method, path, request, response, errors, access); event
names,
                            fields, producer, consumers, topic; versioning (/api/v1)
4. DATA MODEL               tables, keys, Flyway migration plan, tenant/ownership columns if any
5. COMMUNICATION            sync vs async per interaction (with reason); Saga style DECISION and
its
                            switch trigger; outbox or direct publish and the failure window
accepted
6. FAILURE MODES            minimum 5: what fails, how we detect it, what the user sees,
mitigation
                            (include the 2 risks raised in the Architecture Review)
7. SECURITY & DEPLOYMENT    roles per endpoint; secrets; token propagation; images, probes,
Helm,
                            ArgoCD; replaced □ Defaults with justification
8. TEST & LOAD PLAN + RISKS unit/integration/contract tests; coverage target; k6 scenario with
target
                            numbers (P95, req/s); top 5 project risks with owner

# Each decision: DECISION · OPTIONS CONSIDERED · REASON · TRADE-OFF · WHAT WOULD MAKE US REVISIT
```

What a good ADD looks like: a lazy answer says “it is standard, so it is fine.” A good answer says “it is sound, but the risk is X and our mitigation is Y.” An excellent answer says “we would change Z, because the failure mode is …”

10.2 Backlog — 12 to 15 stories

```
# docs/BACKLOG.md — each story:
ID: L2-04   Layer: L2   Gate: G1   Owner: <name>   Estimate: S(≤1h) / M(≤2h) / L(≤4h)
As a <user>, I want <capability> so that <value>.
Definition of Done: <observable check, test name, or curl result>
Depends on: L2-01

# Rules: no story larger than 4 hours (split it) · every Layer L1–L6 has at least one story
# every story names a test or a demonstrable check · first owner assigned for L1 today
```

10.3 Other S25 deliverables

- **Three paper drawings** (photographed, in docs/architecture/): (1) service boundaries of your Bonus and how it touches existing services; (2) sequence of the main flow — happy path AND one failure path; (3) where data lives (table → database). Draw before you write the ADD.
- **Team Charter** signed by all three members (docs/TEAM-CHARTER.md).
- **L0 running:** docker compose up healthy, config-server and eureka-server registered, verify-l0.sh passes.

10.4 Deliverables per gate

Gate	You show live	You commit
G1 — S26	L1 + L2: public GET, 401 without token, ADMIN rules, order pre-check via Feign with resilience, idempotent payment	PRs merged with tests; README with the curl commands
G2 — S27	Saga happy path + compensation; DLT; running in a Kubernetes cluster; probes, Secrets, ArgoCD Synced/Healthy	Dockerfiles, CI workflows, Helm charts, manifests
G3 — S28	k6 smoke/load/stress; one bottleneck found and fixed; trace across HTTP + Kafka; Grafana panel	k6 scripts, Performance Report

Gate	You show live	You commit
G4 — S29	Full demo including the Bonus	Final ADD, README, diagrams, slides, final report

11. Grading

The Capstone is 35% of your course grade.

Component	Weight	Criterion	Weight inside
Team	70%	Architecture Quality (ADD, decisions, diagrams, failure modes)	25%
		Implementation + Tests (Core complete + Bonus; coverage; PR quality)	35%
		Deployment + Security (Kubernetes, GitOps, hardening, secrets)	20%
		Presentation (structure, demo, honesty about trade-offs)	20%
Individual	30%	Oral Q&A in Session 29 (each member answers about any part of the system)	60%
		Code contribution — git log evidence across at least 3 different services	40%

Note on Implementation: a complete Core alone can earn at most 85% of the Implementation criterion; the primary Bonus supplies the remaining 15%.

Grade bands: Distinction 90–100 · Merit 75–89 · Pass 60–74 · Retake below 60.

What Distinction looks like: a working Core, a working Bonus, a live deployment, documented performance results, and a member who can explain and defend any part of the system.

How your S25 work is assessed

Criterion	Points	What is checked
ADD completeness (8 sections)	25	All sections present, decisions follow the DECISION/OPTIONS/REASON/TRADE-OFF/REVISIT form
Quality of reasoning	25	Saga decision justified with a switch trigger; every replaced Default has a trade-off; no “because it is standard”
Failure modes	15	At least 5, realistic, with detection + mitigation; includes the Review’s risks
Paper drawings	10	Boundaries, flow with failure path, data location
Backlog realism	15	12–15 stories, ≤ 4 h each, layers/gates mapped, DoD observable
L0 running + repo hygiene	10	verify-l0.sh passes, protected main, template files kept

12. Final Presentation & Individual Oral Q&A (Session 29)

Planned format — the trainer confirms the exact timing before S29.

Part	Content
Architecture	The problem, your boundaries, key decisions (Saga style, replaced Defaults) and the trade-offs you accepted
Implementation	What is built, how the layers map to the requirements, test and coverage evidence
Live demo	Place an order end to end; a failure path (payment failure → compensation); deployment in the cluster; a trace; the k6 result; your Bonus
Lessons learned	What you would change if you started again, and the open technical debt you consciously left
Individual Oral Q&A	Each member is asked about any part of the system, including the decision they wrote about in the S25 retrospective and what happened to it

Everyone in the team must be able to explain every service, not only the one they wrote. Rotate roles and review each other's PRs so that this is true.

13. Session 25 Checklist

Agenda

Time	Min	Activity
10:00	10	Knowledge Check
10:10	40	Project Charter: vision, requirements, architecture, flows
10:50	15	Break
11:05	30	Technology Decision Matrix + Saga style + repo strategy
11:35	30	Layers & gates, Starter Repo, Bonus Feature Cards, grading
12:05	15	Team formation
12:20	30	Lunch
12:50	40	Capstone Architecture Review (5 questions, 8 minutes each)
13:30	65	Team Sprint 0: repo, L0, drawings, ADD, backlog
14:35	15	ADD peer review (A→B→C→A)
14:50	10	Team Retrospective + checkpoint

Peer review checklist (you will review the next team's ADD)

#	Checklist item	✓/ ✗
1	Problem statement names a real user and a measurable pain	
2	Service boundary is justified by data ownership, not by technical layers	
3	API contract lists endpoints, request/response fields, error codes, and versioning	
4	Event schema lists names, fields, producer, consumers, and topic	
5	Every □ Default that is replaced has a written reason and a trade-off	
6	Failure modes: at least 5 named, each with a detection and a mitigation	
7	Idempotency and duplicate delivery are addressed where messages or retries exist	
8	Security: who can call each endpoint; where secrets live; token propagation	
9	Deployment plan and load-test plan exist with a target number, not just intentions	
10	Backlog is realistic against the time budget; scope-cutting order is stated	

Result: **Approved / Approved with changes / Rework.** A team with Rework fixes its ADD before Session 26.

Definition of Done — Session 25 (commit: session-25: capstone-kickoff-add-and-backlog)

- □ Team repository created from the Starter Repo, main protected, trainer added
- □ L0 running: infra healthy, config-server and eureka-server registered (verify-l0.sh passes)
- □ ADD (8 sections) committed under docs/adr/ with the peer-review result
- □ Three paper drawings committed under docs/architecture/
- □ docs/BACKLOG.md: 12–15 stories with owners for L1
- □ Team Charter signed and committed

Before you come to S25 (Pre-Reading)

- Read this handbook completely.
- Refresh the Session 24 Technical Debt Register v2 and your Session 23 reflection notes.
- Make sure Docker Desktop runs with at least 6 GB RAM; Keycloak needs about 60 seconds to start.

Knowledge Check questions you will be asked at 10:00:

- Name two items still OPEN on Technical Debt Register v2. Which will the Capstone blueprint close by design?
- Session 22 closed three debts. Name them and the mechanism used for each.
- Session 20 deleted JwtAuthFilter. What does that tell us about how to build the Capstone?

14. Common Pitfalls & FAQ

Pitfall / Question	Advice
Our scope is bigger than our time	Core + primary Bonus first. Extra features only after both are complete. Use the scope-cutting order.
Our ADD says “the service may fail”	Write five concrete failure modes: what fails, how you detect it, what the user sees, the mitigation.
Can we add Elasticsearch / another database / another broker?	No. New infrastructure needs a named problem in the ADD and is not allowed for the layers marked . Elasticsearch is explicitly out of scope.
One member does everything	Roles rotate at every gate; each member commits in at least 3 services; the trainer reads the git log at every gate.
Merge conflicts in the parent POM / docker-compose	Short-lived branches, CODEOWNERS on shared files, pull main daily, never reformat shared files.
Keycloak or Postgres not healthy	Give Docker 6 GB RAM; wait 60 s; docker compose logs keycloak; the realm import only runs on first start (remove the volume to re-import).
Can we reuse our lab code?	Yes, as a reference — but build in the Starter Repo in the final shape (e.g. Keycloak Resource Server, Flyway, no in-memory stock).
Can we change a Default?	Yes, with a written justification and trade-off in the ADD. Fixed items cannot change.

Session 26 preview — Gate G1

Bring: L1 + L2 working. Read your own ADD Section 3 as if you were the other team. Be ready to answer: which L1 stories are late and what is the recovery plan; which API field are you least sure the other side will parse; where does the Circuit Breaker fallback return in Order → Inventory and what does the customer see.